IAB207: Assignment 2 Declaration

Section 1: Team declaration (functional / non-functional)

Please declare your team details on this submission itself by completing and including one of the two table options below. The details should reflect the same team as defined in your canvas group. It will assist the marker.

Option A:

Functional Team – most teams should use this one		
Canvas team identifier:	**Ass 2: 59**	
Tutor:	Alaa H. A. Abusafia	
Day and time of tutorial:	Monday, 2:00pm – 4:00pm.	
GitHub name and link:	https://github.com/Erix-Senpai/Ampdup	
Full name	Student#	QUT Email
Tzu-Huan (Eric), Hsu	12022926	n12022926@qut.edu.au
Alexander Tolstoff	10754466	n10754466@qut.edu.au
Reuel Pereira	11916591	n11916591@qut.edu.au
Devashree Kadia	11974907	n11974907@qut.edu.au

Section 2: Team contribution statement

Tzu-Huan (Eric), Hsu	8/9 – 2.5 hours: Github linking, getting everyone started on visual studio, discord, Trello (backlog) and have everyone set, transferring the necessary database up on git, worked on getting the website to load as intended. 10/9 – 0.5 hour: Fixed the website loading and organising the database on git. 11/9 – 3 hours: Organised the backlog, User Stories, and templates required for getting started on assignment. 13/9 - 2 hours Figuring out FlaskForms, linking create event form. 19/9 - 6 hours applied functionality to create event page and displaying events. 20/9 - 6 hours Index.html now is able to display events, with events able to display correctly. 22/9 - 2 hours deleted dummy, now can generate list of events in html. 5/10 - 6 hours added database for events, linked events to the database. 7/10 - 2 hours fixing up multiple naming convention. 20/10 - 2 hours error fixing. 22/10 - 3 hours added conditional check for tickets. 24/10 - 3 hours setup view_modes, displaying correct UI to the user. 28/10 - 5 hours ticket cancellation, purchase, as well as ensuring they work properly. 29/10 - 5 hours ticket cancellation, purchase, as well as ensuring they work properly. 30/10 - 2 hours added event cancellation functionality, 31/10 - 6 hours cleaning up all the codes, fixing errors relating to events. 1/11 - 6 hours cleaning up more codes, fixing the remaining errors.
Alexander Tolstoff	Worked on the following project functionalities: Storage of User Details - 5h Text-Based Search – 7h (Including Image Encoding, expansion of search scope, search results page) Sort by Event Field – 5h (Index implementation) Filter Events by Event Type - 4h (Button configuration) Debugging, reformatting – 5h

Reuel Pereira	The tasks that I completed were: User Commenting – 26 hours (Involved first displaying message, connecting to database and debugging). Error 404 and 500 pages and handling code – 3 hours. Dummy Data – 2 hours. Deploying to PythonAnywhere – 5 hours. Event details page editing – 3 hours. Overall Debugging and formatting – 12 hours.
Devashree Kadia	Tasks I completed: Login form and registration – 6 hours • routes • Form validations • Connecting to database Logout feature – 30mins Purchase ticket feature – 1 day (involved lots of debugging) • Routes • Connecting to forms • Connecting to database • Creating a bookings database table Re-creating database with new table – 5hrs Initialised Flask-login manager to support authentication – 2hrs Adding register button to the navbar – 4 hrs (lots of styling issues were encountered) Fixing front-end styling issues • Adjusting html layout and bootstrap components to ensure responsiveness – 6hrs Setting a ticket limit (Limiting tickets to 10 for normal users, and unlimited for event owner) – 4hrs

Section 3: git shortlog > output.txt

Erix-Senpai (13):

Initial commit
Add files via upload
Add files via upload
Delete main.py
Delete task4_pass_data directory
Add files via upload
Create forms.py
Create auth.py
Merge pull request #1 from Erix-Senpai/whatever
Delete Ampdup directory
Delete IAB207 directory

Delete README.md
Fixed the codebase.

FIRST_NAME LAST_NAME (73):
commit
Fixed navbar active links
Added login/registration form
added login and registration forms
created login/registration form
created login & registration form
resolved merge conflicts and added login/registration forms
added session based logout feature
saving changed before pulling main
trying to connect login/register form to database
trying to connect login to database
added session based logout feature
login function adding authentication
implementing login_manager
merged main into D_Week12
New database, different user table
login/register function connected to database
finalised login/register functions
comming changes before switching branches
almost finalised login/register functions
switching branch
switching branches
changing branches
Add local requirements.txt before merging main
saving local changes before creatinf new branch
added @login_reuired to create event and comment functions
Resolved merge conflict in ampdupdb.sqlite (kept local version)
Merge branch 'D_Week14v2'
added @login_required to create events and commenting
Merge branch 'D_Week14v2'
Save current work before merging latest main
purchase ticket feature added; creates order ID as well
fixing merge conflicts
fixing merge conflicts
added purchase ticket feature/ new order table in db
pulled main
pulled main
Updated and pulled main
Merge branch 'main' of https://github.com/Erix-Senpai/Ampdup into D_Week14v5
fixed ticket quantity dropdown
pulled main
merged main into my branch
added register button

added register button, pulled main
Merge branch 'main' of https://github.com/Erix-Senpai/Ampdup
Merge branch 'D_Week14v5'
register button added
pulled main
fixed registration warnings
Merge branch 'main' of https://github.com/Erix-Senpai/Ampdup into D_Week14v5
Merge branch 'main' of https://github.com/Erix-Senpai/Ampdup
commiting before switching branches
added registrations warnings
addedd proper registration warnings
commiting before switching branches
Merge branch 'main' of https://github.com/Erix-Senpai/Ampdup
Merge branch 'main' into D_Week14v5
fixing home page layout
pulled main
pulled main
Merge branch 'main' into D_Week14v5
fixed some styling issues
fixed some styling issues
Merge branch 'main' of https://github.com/Erix-Senpai/Ampdup
pulled main
pushed to main
commiting before pulling main
pulled main
pull main
pulled main
new branch
up to date with main
added ticket limit feature

Reuel Pereira (44):
connected booking history.html
testing commit
Finished adding base content
Fixed logo navigation
Fixed title
Merge branch 'main' of https://github.com/Erix-Senpai/Ampdup
Finished comment update and view
Merge branch 'main' of https://github.com/Erix-Senpai/Ampdup
secret key
Changed createEvent naming
Merge branch 'main' of https://github.com/Erix-Senpai/Ampdup
added db for comments
Merge branch 'Reuel5' of https://github.com/Erix-Senpai/Ampdup
Entered stuff in database
added db for comments

Entered stuff in database
moved instance into codebase
Added strftime and comments show username
Added Error 404 and 505 pages
adjusted file layout for pythonanywhere
Attempted adjustment for layout
edited the database to have phone number not be unique
Added a little dummy data to database
made sure footer stays at the bottom of the page.
added a couple comments to events.py
Merge branch 'main' of https://github.com/Erix-Senpai/Ampdup
Added to database
debug
Merge branch 'main' of https://github.com/Erix-Senpai/Ampdup
Edited the comment format
Made event card images remain the same height
Merge branch 'main' of https://github.com/Erix-Senpai/Ampdup
Fixed category image card look
Merge branch 'Reuel15'
Added to database and fixed search results returning 404 error when view details is pressed
replaced event description from the cards with price
main merge
Merge branch 'main' of https://github.com/Erix-Senpai/Ampdup
Changed booking history card display
Status cannot be changed by user when creating the event
Cleared a few files
Merge branch 'main' of https://github.com/Erix-Senpai/Ampdup
Fixed the comments
Added comments to database

ReuelPereira (2):
Delete __pycache__ directory
added requirements.txt for pythonanywhere

alex (18):
User Detail getting stored properly
Final w10 commit
Merge branch 'main' of https://github.com/Erix-Senpai/Ampdup
Fixed main merge
Base Implementation of Text-Based Search Functionality
real main with real db
DB Update
Images display for search results
Merge branch 'alex-w13-sort'
Handling for no Search Results
Merge branch 'alex-w13-sort'

Sorting Functionality Basically Complete
Event Type Filtering Complete
Less ugly button
Misc Commenting
Minor Fix
Minor Fixes for Search Result Display
Final Modifications to Search, Sort and Filter

amps (4):
db work
Merge branch 'alex-w13-speedrun'
db fix
Merge branch 'alex-w13-speedrun'

erix-senpai (61):
Testing commit
Fixed the site linking via changing the hardcode href to the @main.bp function via {{ url_for('main.__filename__') }}. Additionally, filename for "Event Details" is changed to "Event_Details", html file is not meant to have spacings.
Create_Event Form on Submit now returns to home page.
commit
Fix Merge to Main Test 1
Temporarily deleted pycache files.
Event Details in relavence to the index.html page now updates to based on the example data implemented inside test.py. Additionally, test.py index_database.py will return data that relate to the home page, specifically the events. This will still require database wiring.
Added the Ticket Cost to event.py. Inside index.html, the events now runs through a forloop to generate the list of events.
Merge branch 'eric_week9.5'
Created a separate file to merge main with eric_week9.5 to main.
Changed event.py to create_events.py to avoid naming convention collision.
week 10 eric branch
Added database file, added db for event, create_event now generates into the database.
Added database to creating event. Additionally, index_database, which loads the events ongoing in index.html, now loads all the events within the Event database.
Merge branch 'eric_week10.5'
test
added correct database to main
correctly merge eric database to main
temp. commit to checkout past files to fix the database in relation to event db.
Moved event_db to models. Merged class 'Event'. Renamed the Event table's data, as well as the naming convention. Event_Details.html now will load the relavent event correctly and only the relavent event. Foreign Key 'Event' has been added to comments.
Moved event_db to models. Merged class 'Event'. Renamed the Event table's data, as well as the naming convention. Event_Details.html now will load the relavent event correctly and only the relavent event. Foreign Key 'Event' has been added to comments.
Clear cache.

week 10 eric branch

Added database file, added db for event, create_event now generates into the database.

Added database to creating event. Additionally, index_database, which loads the events ongoing in index.html, now loads all the events within the Event database.

correctly merge eric database to main

temp. commit to checkout past files to fix the database in relation to event db.

Moved event_db to models. Merged class 'Event'. Renamed the Event table's data, as well as the naming convention. Event_Details.html now will load the relavent event correctly and only the relavent event. Foreign Key 'Event' has been added to comments.

Clear cache.

Successfully merge D's work to main.

Properly Merged D's work to main.

Fixed the duplication error in Event_Details.html

Modified event related methods and functions to parse through tickets, ticket remaining, and parses through time to check for whether if the status should be changed to inactive after due date. Also added ticket remaining on event_details.html, bookinghistory functionality (still in progress.)

Merge branch 'main' of https://github.com/Erix-Senpai/Ampdup

Completion of Booking Event Functionality. Securely linked to user, with relavent user having access to cancelling the event.

Merge branch 'eric_week13-4' to main.

Fixed D's Purchase Ticket funct. Added limitation to bookings and ticket purchase amount.

Fixed some bugs relating to order cancellation.

Merge branch 'eric_week13-5'

create_eric_week14

Ticket Purchase/Cancellation now updates to database properly, and can be cancelled properly. Additionally, certain functionalities are disabled as intended upon satisfying a requirement.

Added functionality for Event Cancellation. Added func to show ticket remain, as well as enabling and disabling cancellation rules.

Merge branch 'eric_week14'

Merge eric_week14 to main.

Copy of Main with edits.

Fixed search function within view.py. Modified view.py's code structure to compact the code. Added @login_required for multiple functions across the python code. Modified BookingHistory.html and Event_details.html to display the neccesary details correctly.

Modified style.css to adjust the naming. Changed the box grids for index.html. Fixed the Button Sizing.

Merge branch 'eric_week14-5'

Added Comments. Cleaned up files. Simplified some codes where possible.

Added comments. Debugging event button functionality, including cancel booking, cancel event.

Split the cancellation file of cancel booking and cancel event to individual routing. Added parameters to check if user is truely authenticated. Fixed event cancellation button, and booking cancellation.

Merge branch 'main' of https://github.com/Erix-Senpai/Ampdup

Merge branch 'eric_week14-6'
Deleted Cache. Added dummy data.
Fixed minor error in sql.
Fixed Auto-Sold out button.
Merge branch 'eric_week14-6'
Merge eric_week14-6 to main
Fixed Sold Out button functionality.
When user cancel's booking, the event of which the original status set as 'sold out', will now change to 'open'.
Merge branch 'main' of https://github.com/Erix-Senpai/Ampdup

strxwbxrryy (1):
Add files via upload

Section 4: Team use of AI statement

Part B:
As a team, we report that some members have stated they AI in the following authorized way.

Summary of AI use
(no word limit but we suggest 50-100 words describing how your team used AI)

Key examples of AI use
Your marker needs to clearly understand how your team used AI to improve your work. Provide representative samples of how you used AI using the requested format.

- **Paste the prompt** you gave to the AI (e.g. "Improve the grammar and spelling in the attached document without changing any of the meaning").

How can I make this footer stick to the bottom of the page?

- **Paste any input** or upload that accompanied the prompt (e.g. the document, text, or context you provided).

<footer class="bg-body-secondary py-3 mt-auto ampdupFooter">

<ul class="nav justify-content-center border-bottom pb-3 mb-3">

```
            <li class="nav-item"><a href="#" class="nav-link px-2 text-body-secondary">Home</a></li>

            <li class="nav-item"><a href="#" class="nav-link px-2 text-body-secondary">Venues</a></li>

            <li class="nav-item"><a href="#" class="nav-link px-2 text-body-secondary">Support</a></li>

            <li class="nav-item"><a href="#" class="nav-link px-2 text-body-secondary">FAQs</a></li>

            <li class="nav-item"><a href="#" class="nav-link px-2 text-body-secondary">About</a></li>

          </ul>

          <p class="text-center text-body-secondary">© 2025 AmpdUP, Inc</p>

        </div>

      </footer>
```

- **Paste the output** generated by the AI.

To make your **footer stick to the bottom of the page**, you can use **CSS Flexbox** or **Bootstrap utility classes** — depending on how your overall HTML is structured.

Here are **three reliable approaches**, depending on your setup 👇

✅ Option 1: Modern Flexbox Layout (Recommended)

This is the cleanest and most robust approach.

Add this to your main CSS (e.g., style.css):

```
html, body {
  height: 100%;
  margin: 0;
}

body {
  display: flex;
  flex-direction: column;
```

```
  min-height: 100vh;
}

footer {
  margin-top: auto;
}
```

Ensure your HTML structure looks like:

```
<body>
  <header>...</header>

  <main class="flex-grow-1">
    <!-- Your main content -->
  </main>

  <footer>
    <div class="container bg-body-secondary">
      <footer class="py-3 my-4 ampdupFooter">
        <ul class="nav justify-content-center border-bottom pb-3 mb-3">
          <li class="nav-item"><a href="#" class="nav-link px-2 text-body-secondary">Home</a></li>
          <li class="nav-item"><a href="#" class="nav-link px-2 text-body-secondary">Venues</a></li>
          <li class="nav-item"><a href="#" class="nav-link px-2 text-body-secondary">Support</a></li>
          <li class="nav-item"><a href="#" class="nav-link px-2 text-body-secondary">FAQs</a></li>
          <li class="nav-item"><a href="#" class="nav-link px-2 text-body-secondary">About</a></li>
        </ul>
```

```
        <p class="text-center text-body-secondary">© 2025 AmpdUP, Inc</p>
      </footer>
    </div>
  </footer>
</body>
```

💡 **Why it works:**

- The body fills the viewport (min-height: 100vh).
- footer { margin-top: auto; } pushes the footer to the bottom if content is short.

✅ **Option 2: Using Bootstrap Utility Classes Only**

If you prefer not to write custom CSS, wrap your layout like this:

```
<body class="d-flex flex-column min-vh-100">
  <main class="flex-grow-1">
    <!-- Your main content here -->
  </main>

  <footer class="mt-auto bg-body-secondary py-3">
    <div class="container text-center">
      <ul class="nav justify-content-center border-bottom pb-3 mb-3">
        <li class="nav-item"><a href="#" class="nav-link px-2 text-body-secondary">Home</a></li>
        <li class="nav-item"><a href="#" class="nav-link px-2 text-body-secondary">Venues</a></li>
        <li class="nav-item"><a href="#" class="nav-link px-2 text-body-secondary">Support</a></li>
        <li class="nav-item"><a href="#" class="nav-link px-2 text-body-secondary">FAQs</a></li>
        <li class="nav-item"><a href="#" class="nav-link px-2 text-body-secondary">About</a></li>
```

```
        </ul>
        <p class="text-center text-body-secondary">© 2025 AmpdUP, Inc</p>
      </div>
    </footer>
</body>
```

💡 Here:

- d-flex flex-column min-vh-100 makes the body full height and vertically stacked.
- mt-auto on the footer pushes it to the bottom.

🚫 **Avoid: position: fixed; bottom: 0;**

You *can* use this for a permanently visible footer, but it will **overlay** content if your page scrolls:

```
footer {
  position: fixed;
  bottom: 0;
  width: 100%;
}
```

Only use this if you want a *persistent* footer that's always visible.

Would you like it to **always be visible (sticky)** or only **appear at the bottom when scrolling ends**?
I can tailor the CSS for that behavior.

========== NEXT SAMPLE OF AI USE ==========

Intention: Don't know the code to use for writing Sql-Alchemy in python. Searching for ways to call events/Booking from the database, where I don't want all booked events, but filtered queries.

how do I filter through the event I want to call with calling for the database in python? Say I first want to get all the event that's booked by the user. Then, I also want to get all

the booked event that is the user's own hosted event. But, when getting booked event, it gets all of them.

That's an excellent question — and it's a very common situation in event-booking systems.
You essentially have **two relationships** with the `Event` table:

1. Events that the user **booked** (through the `Bookings` table).
2. Events that the user **owns** (from the `owner_id` in `Events`).

Let's break it down carefully 👇

🧩 Your models (simplified)

You have something like:

```
class Event(db.Model):
    __tablename__ = "Events"
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String)
    owner_id = db.Column(db.Integer, db.ForeignKey('Users.id'))
    # ... other fields ...

class Booking(db.Model):
    __tablename__ = "Bookings"
    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey('Users.id'))
    event_id = db.Column(db.Integer, db.ForeignKey('Events.id'))
```

☑️ Case 1: All events booked by the user

You already have this one:

```
booked_events = (
    db.session.execute(
        db.select(Event)
        .join(Booking, Booking.event_id == Event.id)
        .where(Booking.user_id == current_user.id)
    ).scalars().all()
)
```

That returns all events where the **user booked** them, even if they didn't host them.

☑ Case 2: All events the user *owns* (hosted events)

That's simpler — it just looks at the `owner_id` in the `Events` table:

```
owned_events = (
    db.session.execute(
        db.select(Event)
        .where(Event.owner_id == current_user.id)
    ).scalars().all()
)
```

☑ Case 3: Booked events *that the user owns* (hosted and booked)

Now this is the combination: you want events that are **booked by the user AND owned by the same user**.

You can chain both conditions:

```
booked_owned_events = (
    db.session.execute(
        db.select(Event)
        .join(Booking, Booking.event_id == Event.id)
        .where(
            Booking.user_id == current_user.id,
            Event.owner_id == current_user.id
        )
    ).scalars().all()
)
```

This returns only those events where:

- The user booked the event (`Booking.user_id == current_user.id`), and
- The user is also the owner of that event (`Event.owner_id == current_user.id`).

☑ Putting it together (example)

You can combine all of these in one route if you like:

```
@booking_history_bp.route('/', methods=['GET', 'POST'])
@login_required
def Get_Booking():
    # 1️⃣ Events the user booked
    booked_events = db.session.execute(
```

```
        db.select(Event)
        .join(Booking, Booking.event_id == Event.id)
        .where(Booking.user_id == current_user.id)
    ).scalars().all()

    # 2️⃣ Events the user owns
    owned_events = db.session.execute(
        db.select(Event)
        .where(Event.owner_id == current_user.id)
    ).scalars().all()

    # 3️⃣ Events that the user both owns *and* booked
    booked_owned_events = db.session.execute(
        db.select(Event)
        .join(Booking, Booking.event_id == Event.id)
        .where(
            Booking.user_id == current_user.id,
            Event.owner_id == current_user.id
        )
    ).scalars().all()
return render_template(
    'BookingHistory.html',
    booked_events=booked_events,
    owned_events=owned_events,
    booked_owned_events=booked_owned_events
)
```

✅ Example in the template You can now separate the sections visually:

html Copy code

Booked Events

{% for event in booked_events %}

{{ event.title }}

{% else %}

No booked events.

{% endfor %}

Owned Events

{% for event in owned_events %}

{{ event.title }}

{% else %}

No owned events.

{% endfor %}

Owned & Booked Events

{% for event in booked_owned_events %}

{{ event.title }}

{% else %}

No self-booked events.

{% endfor %} Would you like me to show how to merge these queries efficiently into one (e.g. for filtering with buttons in the UI, like “Show all / Owned / Booked”)? That’s a nice optimization step if you plan to add filters or tabs.

========== NEXT SAMPLE OF AI USE ==========

Use of AI: (Devashree)
Intention: I didn’t know how to query the database for the user’s booked tickets for purchasing tickets feature.

Prompt: how to get an integer value of the number of tickets a user has bought for an event from a database

AI Response:

1. Assumptions about your database

You likely have a table that tracks ticket purchases, e.g., `orders`:

Column	Description
id	unique order ID
user_id	the user who bought tickets
event_id	the event the tickets are for
quantity	how many tickets were bought in that order

2. SQL query approach

If you just want the total tickets a user bought for an event:

```
SELECT SUM(quantity) AS total_tickets
FROM orders
WHERE user_id = ? AND event_id = ?;
```

- Replace ? with the user's ID and event ID.
- `SUM(quantity)` adds up all tickets across multiple orders.
- If the user hasn't bought any tickets, the result will usually be `NULL`, which you can treat as 0.

3. Python / SQLAlchemy approach

Assuming you're using SQLAlchemy:

```
from sqlalchemy import select, func

# total tickets user has purchased for this event
total_tickets = db.session.scalar(
    select(func.sum(Order.quantity))
    .where(Order.user_id == current_user.id)
    .where(Order.event_id == event.id)
)

# Ensure we get an integer
total_tickets = int(total_tickets or 0)

print(total_tickets)  # e.g., 5
```

- `func.sum(Order.quantity)` sums all the quantities.
- `db.session.scalar()` returns a single value (or None if no rows).
- `int(total_tickets or 0)` ensures it's always an integer.

4. Python / raw SQL approach

If you're using raw SQL (e.g., SQLite or MySQL):

```
user_id = 1
event_id = 42

cursor.execute("""
    SELECT SUM(quantity) FROM orders
    WHERE user_id = ? AND event_id = ?
""", (user_id, event_id))
```

```
result = cursor.fetchone()[0]  # fetch the sum
total_tickets = int(result or 0)

print(total_tickets)
```

- Works in any Python DB connection.
- `result` is None if no tickets purchased, so we default to 0.

✅ **Summary**

- Use `SUM(quantity)` on your orders table for that user and event.
- If using Python, convert None to `0`.
- Result is an integer that tells you exactly how many tickets the user has already purchased.

Document version history:

Date	Version	Changes made
25/08/25	V1	First version